

Infrastructure as Intent (IAI)

Whitepaper I: Concept and Architecture

First in a three-part whitepaper series on Infrastructure as Intent

Elaia Raj

2026

Abstract

Infrastructure has become easier to provision, but not necessarily easier to interpret. The industry has moved from physical servers to virtualization, cloud, and Infrastructure as Code. Each stage improved execution. However, the hardest part often remains unchanged: a person still has to translate business intent into infrastructure decisions.

This translation requires technical judgement, security awareness, cost awareness, platform knowledge, and understanding of organizational standards. In many organizations, this knowledge lives mainly in the minds of experienced engineers. The result is slow delivery, inconsistent interpretation, and fragile dependency on individuals.

Infrastructure as Intent, or IAI, proposes a layer above existing infrastructure tools. A human states the desired outcome in plain language. An agent reads a declared source of truth, selects the right execution engines, generates the required artefacts, validates the change for correctness, security, and cost, and then presents an approval summary in a plain language.

This is possible now in a way that was not possible a few years ago. AI agents can now keep context, reason through several steps, and operate within clear limits. This means they can be trusted with controlled infrastructure tasks, provided that the right governance is in place. The same intent layer should not only manage public cloud. It should also cover on-premise virtualization and the physical hardware that sits beneath it.

IAI is not a replacement for Terraform, CloudFormation, Kubernetes, GitOps, or Policy as Code. It is the reasoning layer above them. Its purpose is to make infrastructure translation explicit, repeatable, governed, and auditable.

1. Introduction and Thesis

Infrastructure as Code made infrastructure repeatable and auditable. But it did not remove the need for a human to understand the business requirement, decide what it means technically, choose the right tools, and write the code. The code is not the beginning of the process. It is the output of reasoning that already happened inside a person's head.

This is the gap IAI addresses. Most automation focuses on execution. Cloud platforms can provision quickly once the instruction is clear. IaC tools can apply configuration once the code exists. Pipelines can validate once artefacts are produced. The slower and more fragile part is often the translation that happens before the machine is asked to do anything.

The thesis of this paper is simple: the next stage of infrastructure automation is not just writing infrastructure code faster. It is formalizing the reasoning layer that exists before code is written. The human should describe the desired outcome. The system should translate, validate, explain, and seek approval before execution.

2. The Translation Tax

A business request rarely arrives as a technical specification. It may say: “Create a secure production environment for a new customer-facing service.” That statement is intent. It still has to be translated into networks, compute, identity, storage, encryption, monitoring, backup, tagging, logging, cost controls, and deployment rules.

This hidden work is the translation tax. Every infrastructure request pays it before provisioning can begin.

A typical version of this request: a staging environment for a new service holding customer data, needed within two weeks. The request itself is short. What it implies is not — encryption at rest, a backup posture, a data-residency choice, and a tagging convention a handful of engineers carry by memory rather than by document.

- It appears as lead time because requests wait for the right engineer or architect.
- It appears as inconsistency because different people interpret the same requirement differently.
- It appears as risk because standards and exceptions may live in memory rather than systems.
- It appears as cost leakage because financial impact is often considered too late.
- It appears as organizational fragility because critical knowledge leaves when people leave.

IaC did not remove this tax. In some environments, it increased it because the translator must now understand multiple tool syntaxes, provider behaviours, internal standards, security controls, and approval processes. AI coding assistants may reduce typing, but they do not solve the organizational translation problem if each engineer prompts separately and receives different answers.

3. Defining Infrastructure as Intent

Infrastructure as Intent is a model where an agent receives a human infrastructure intent, interprets it against a declared source of truth, selects and operates the appropriate execution engines, validates the proposed change, and maintains the resulting infrastructure knowledge under human approval.

The agent is not the execution engine. Terraform, CloudFormation, Kubernetes, APIs, network automation, and other tools continue to do what they already do well. The agent sits above them. It decides which engine should be used, what artefacts are required, what rules apply, and how the proposed change should be explained.

The key separation is this:

Layer	Purpose
Intent Layer	Understands meaning, context, policy, criticality, business purpose, and tool selection.
Code Layer	Executes explicit instructions through IaC tools, cloud APIs, platform APIs, and automation engines.

This separation protects existing investment. Organizations do not need to throw away their current tools. IAI treats those tools as engines within a wider reasoning and governance model.

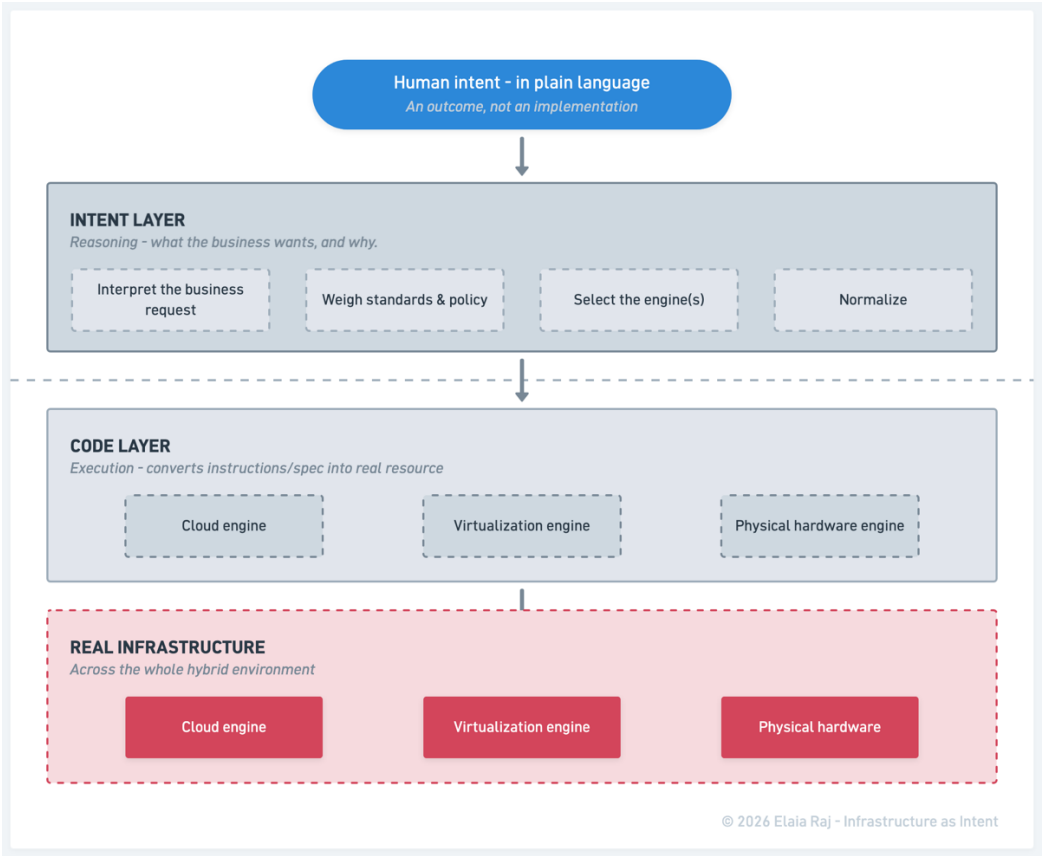


Figure 1. The intent layer sits above the code layer, which in turn operates the real infrastructure across the cloud, virtualization, and physical hardware engines.

4. What IAI Is Not

IAI should not be confused with a natural-language wrapper around Terraform. Such a wrapper may generate code, but the human may still need to know what to ask for, which tool to use, which policy applies, and whether the output is safe.

IAI is also not merely a copilot. A copilot helps an engineer work faster. IAI changes the starting point. The operator begins with the desired outcome, not the implementation method.

IAI is not a replacement for IaC, GitOps, Policy as Code, AIOps, FinOps, or platform engineering. These remain important. IAI connects them around intent. Its value is not that it hides code. Its value lies in making the translation from business intent to infrastructure execution explicit, consistent, auditable, and governed.

5. Reference Scenario

Consider this request: *“Create a public-facing campaign environment for a new product launch. It must support high traffic, stay within budget, be monitored, and be removed after thirty days.”*

In a traditional model, an engineer or architect translates this manually. In an IAI model, the agent interprets the intent against the Platform Manifest and organizational standards.

Intent Signal	Possible Infrastructure Interpretation
“New campaign environment”	Requires environment creation based on an approved temporary workload reference template, not a design invented from scratch.
“Public-facing”	Requires an approved internet-facing entry point, WAF rules, TLS certificates, DNS configuration, access logging, and exposure review.
“High traffic”	Requires load balancing, autoscaling, performance thresholds, capacity assumptions, and monitoring for traffic spikes.
“Stay within budget”	Requires cost estimation, resource size recommendation, budget alerts, and approval if the expected spend exceeds threshold.
“Monitored”	Requires health checks, logs, metrics, alerts, dashboards, and ownership assignment from the beginning.

Intent Signal	Possible Infrastructure Interpretation
“Removed after thirty days”	Requires expiry tagging, scheduled teardown, data lifecycle decisions, owner notification, and manifest update after removal.

The output should not be raw code only. It should be a decision briefing: what will be created, why those choices were made, what policies were applied, what assumptions remain, what it will likely cost, when the environment will expire, and what the teardown path is.

This scenario shows the practical value of IAI. The environment is new, but the pattern is known. The human begins with the desired outcome, while the system translates that intent into governed execution.

6. End-to-End Architecture

The IAI pipeline can be described in eight stages:

1. **Intent capture** - a human describes the desired outcome in plain language. The request states what is needed and why, not which tools to use or which resources to create.
2. **Manifest reading** - the agent reads the Platform Manifest as the source of truth. As a result, the interpretation is grounded in declared facts, standards, and criticality levels rather than assumptions.
3. **Tool selection** - the agent chooses the right execution engines for the request. The choice comes from the manifest, which declares which engine manages each environment, so the decision is transparent and reviewable rather than guessed.
4. **Artefact generation** - the agent creates the required IaC, manifests, policies, or API calls. In other words, the agent pays the translation tax that an engineer would otherwise pay by hand, and it applies organizational standards consistently every time.
5. **Validation gates** - correctness, security, and cost are checked before approval. Each gate runs independently of the generation step, so the proposed change is verified rather than assumed to be correct.
6. **Approval summary** - the agent explains the proposed change in plain language. The human receives a single synthesized briefing covering what will change, what it will cost, and what was caught, instead of raw tool output.
7. **Apply** - after approval, the agent executes through the selected engine. Nothing is applied before the human decision, and state is captured first so the change can be rolled back if needed.
8. **Manifest update** - the agent records what changed and why, keeping knowledge current. As a result, documentation and reality move together instead of drifting apart over time.

When no request is pending, the same layer can monitor drift, compliance posture, cost movement, and health signals. This means IAI is not only used to provision infrastructure. It also helps keep the infrastructure aligned, controlled, and continuously visible after it has been created.

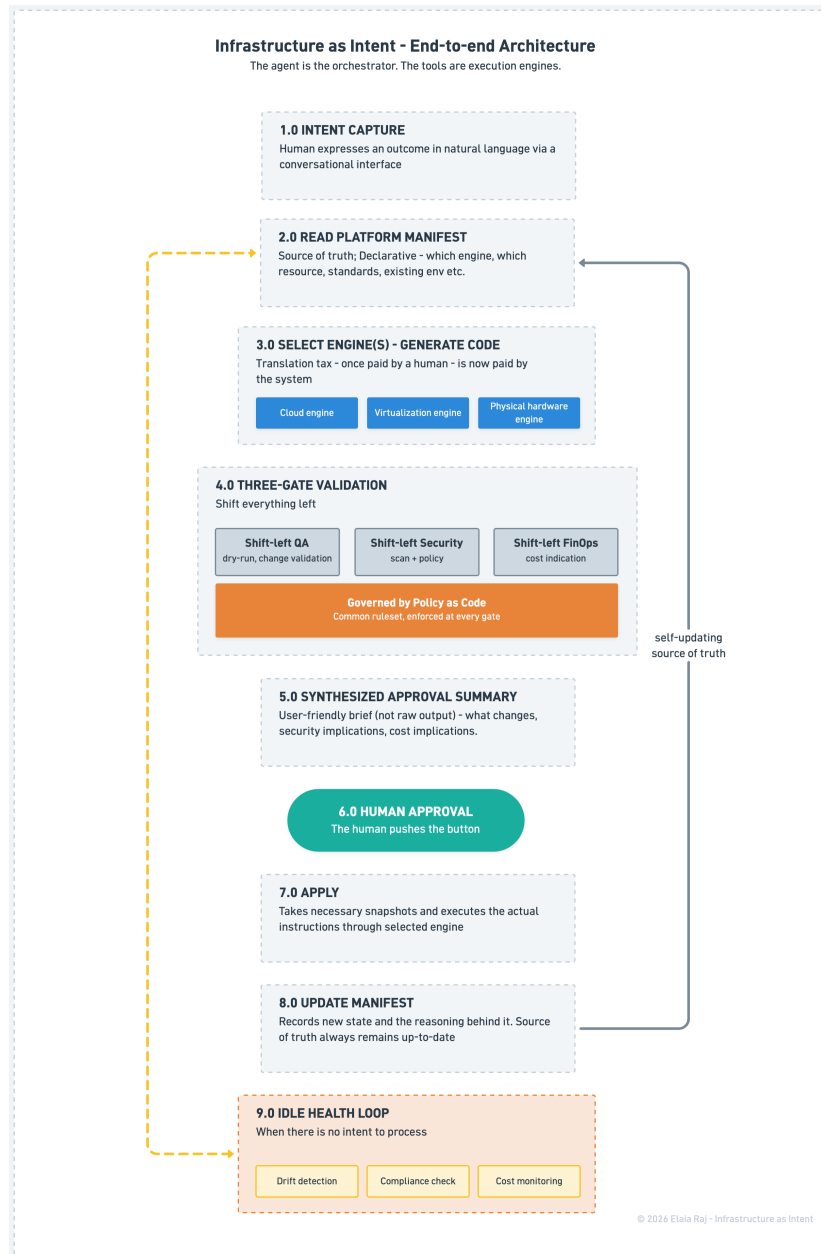


Figure 2. End-to-end architecture, from intent capture through three-gate validation and approval to manifest update, with continuous monitoring during idle periods.

7. Design Pillars

7.1 The Self-Maintaining Platform Manifest

The Platform Manifest is the declared source of truth. It records what exists, which engine manages it, what business purpose it serves, what criticality level it carries, what policy applies, and why design decisions were made.

This matters because documentation often becomes stale when it is separate from the work. In IAI, the agent that changes the infrastructure also updates the manifest. Over time, the manifest becomes a living architecture decision record.

7.2 Load-Bearing Dependencies

Not all infrastructure carries the same risk. A development sandbox and a production payment service should not be treated equally. IAI requires criticality to be declared and propagated through dependencies.

If a business-critical service depends on a database, network path, identity provider, or storage bucket, those components inherit importance. This makes organizations protect the full dependency chain, not just the visible application.

7.3 Shifting Everything Left: Policy as Code and the Three Gates

IAI uses policy to validate every proposed change before execution. The three gates are correctness, security, and cost.

Gate	Purpose
Correctness	Checks whether the proposed change is technically valid and can be planned or applied by the selected engine.
Security (SecOps)	Checks encryption, exposure, identity, compliance baselines, and approved patterns.
Cost (FinOps)	Estimates likely financial impact before spend is committed.

The cost gate does not need to predict the exact final bill. Its value is to create informed decisions before deployment rather than surprises after the bill arrives.

7.4 Explainability and Auditability

The agent must explain why it selected a design, what policies were applied, what alternatives were rejected, who approved the change, and what changed after execution. This makes the decision path visible to engineers, architects, auditors, and business stakeholders.

7.5 Rollback Discipline

Any system that can change infrastructure must treat rollback as a first-class concern. Before applying a change, the agent should capture state and, where relevant, data snapshots. It must also state what can be rolled back automatically and what cannot.

8. Governance, Accountability, and Trust Boundaries

IAI does not remove responsibility. It changes where responsibility is applied. The human remains accountable for approving meaningful change. The agent is responsible for consistent translation, validation, explanation, execution, and manifest maintenance.

A practical IAI model should define:

- Who owns the original intent.
- Who approves changes for each environment and criticality.
- Who owns the Platform Manifest.
- How exceptions and policy overrides are recorded.
- How rejected recommendations become feedback.
- What the agent is allowed to plan, propose, and apply.

The agent should operate with least privilege. Planning and applying should be separated. Production access should be tightly scoped. Audit logs should be immutable. Autonomy should be limited by environment, risk, and execution history.

9. The Autonomy Model: Earned Trust

The most sensitive part of IAI is autonomy. The model should begin with human approval for every change. This is not a weakness. It is a necessary control because infrastructure decisions have operational, security, and financial consequences.

Trust should be earned through repeated successful behaviour. Low-risk, repetitive, well-understood changes may eventually be applied automatically within strict guardrails. Higher-risk changes, especially those affecting production, sensitive data, or business-critical systems, should continue to require human approval.

The aim is not to remove human judgement. The aim is to reserve human judgement for decisions that are highly consequential.

10. IAI Maturity Model

An organization need not attempt full autonomy on the first day. This model describes each stage, allowing organizations to progress at their own pace as the system proves itself reliable.

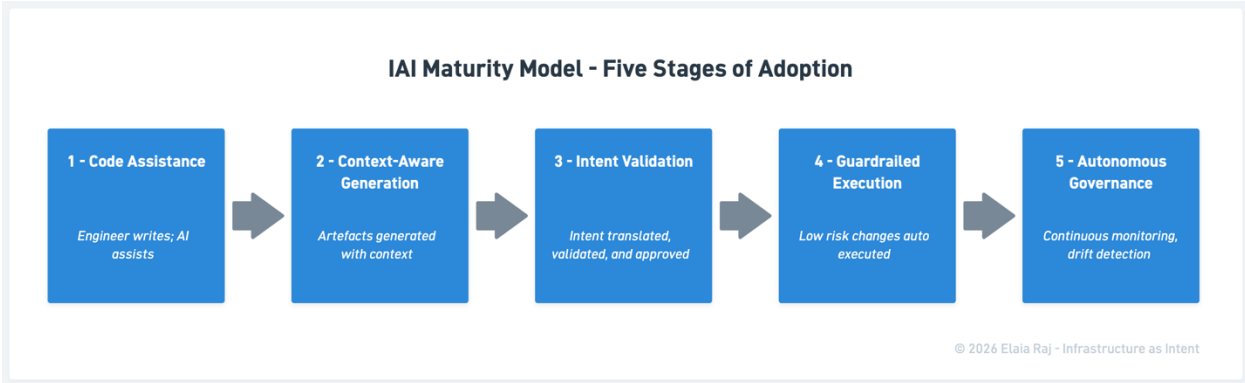


Figure 3. IAI Maturity Model; Organizations progress from code assistance to autonomous governance as system reliability grows and human oversight decreases.

Level	Stage	Description
1	Code Assistance	Engineers are assisted in writing, reviewing, or explaining infrastructure code, while they remain responsible for interpretation, design decisions, and implementation.
2	Context-Aware Generation	Infrastructure artefacts are generated using declared standards, platform context, and organizational conventions, while engineers remain responsible for validation and final decisions.
3	Intent Validation	Infrastructure intent is translated into proposed changes and validated for correctness, security, and cost, while human approvers remain responsible for authorizing execution.

Level	Stage	Description
4	Guardrailed Execution	Low-risk, repetitive infrastructure changes are executed within defined guardrails, while higher-risk or business-critical changes continue to require explicit human approval.
5	Autonomous Governance	Infrastructure posture is continuously monitored, drift is detected, improvements are proposed, the manifest is maintained, and approved changes are governed within defined policy boundaries.

11. Measuring Success

IAI should be measured by operational outcomes. Useful measures include:

- Reduced infrastructure request lead time.
- Fewer policy violations before production release.
- Improved tagging, naming, encryption, backup, and logging compliance.
- Lower dependency on individual senior engineers for routine translation.
- Better cost visibility before deployment.
- Improved audit readiness through decision history and manifest updates.
- Reduced drift between documentation and real infrastructure.

The value of IAI is not only speed. It is consistency, resilience, and preservation of knowledge base within the organization.

12. Current Industry Position

Many parts of IAI already exist, but they are fragmented. IaC tools provision resources. GitOps manages deployment flow. Policy as Code enforces rules. FinOps tools estimate and monitor cost. AIOps tools analyze operational signals. Platform engineering provides self-service interfaces. AI coding assistants generate code.

IAI is different because it makes intent the orchestrating layer. The execution engines remain specialized, but the reasoning layer becomes unified. This is the gap the model addresses.

This is already visible in the market: vendors have begun shipping natural-language, intent-based interfaces to infrastructure platforms as production features rather than research previews. What will separate a durable intent layer from an interface feature is the ability to work across cloud, on-

premise virtualization, and physical infrastructure, earn trust over time, and keep its own source of truth updated.

13. Risks and Limits

IAI must be approached carefully, nevertheless. The model should not assume that an agent is always correct. It should assume that an agent can become useful when its reasoning is constrained by a source of truth, checked by policy, and supervised by humans.

- Ambiguous intent can lead to incorrect interpretation. The agent must ask for clarification when important details are missing.
- Generated artefacts may look correct while still violating an architectural principle. Independent validation gates are necessary.
- Cost estimates are useful signals, not guarantees.
- The Platform Manifest can become a risk if it drifts from reality.
- Rollback is not always complete, especially where data-bearing systems or application-led schema changes are involved.
- Too much autonomy too early can create operational and security risk.

14. Forward Vision

The first version of IAI should be manifest-driven because that makes it easier to govern. Over time, the model can become more capable. It may discover unmanaged infrastructure, identify undocumented relationships, propose manifest updates, and recommend improvements before a human asks.

Reusable intent patterns may also emerge. Examples include “secure production web service,” “regulated customer environment,” “cost-optimized dev sandbox,” and “temporary testing environment.” These patterns would allow organizations to express common infrastructure outcomes consistently without rebuilding the reasoning each time.

The longer-term significance of IAI is not only automation. It is the conversion of organizational infrastructure knowledge into a tangible, durable and active asset.

15. Conclusion

The history of infrastructure has been a history of moving manual effort upward. Physical servers required manual setup. Virtualization reduced hardware dependency. Cloud reduced fixed capacity dependency. Infrastructure as Code reduced manual provisioning. Each stage automated more of the lower layer and exposed the next layer of human effort.

The remaining layer is translation. It is the point where a human understands what the business wants and converts that understanding into infrastructure.

Infrastructure as Intent formalizes that layer. It does not remove infrastructure tools or engineering judgement. It changes where judgement is applied. The human focuses on the decision. The agent performs translation, validation, explanation, execution, and knowledge maintenance.

The future of infrastructure management will not be defined only by who can write infrastructure code faster. It will be defined by who can preserve intent, apply judgement consistently, and turn organizational knowledge into an active system.

Organizations that begin building this layer now compound an advantage that becomes difficult to copy later: a record of decisions that grows more complete, and more accurate, with every change it governs.

Infrastructure as Intent is the intellectual property of Elaia Raj. This whitepaper is part of an ongoing series.